H1 Hands-on · Slicing and Anti-Patterns

Goal: two diffs side by side
30 minutes

REWE DIGITAL

Hands-on 1: What it’s about

30 minutes — ticket **`issues/01-products-filterable.md`**

The idea: Every implementation involves decisions. Either you make them, or Claude guesses them plausibly. You make this visible: Claude implements the same ticket twice.

Run 1 · Just let it get done

Hand over the ticket unchanged, don’t intervene.
If Claude asks back: “Decide for yourself.”
Claude decides everything open on its own

Run 2 · Clarify first, then implement

Claude lists all open points with a recommendation.
You adopt the recommendations as your decisions.
Task in four building blocks, then implementation

ONLY THE NUMBERS ARE COMPARED: DON’T READ CODE, DON’T JUDGE THE SOLUTION

Files / lines	AI decisions	of which recorded	`verify green? · tests`

Stuck? The playbook contains all prompts and commands. Try it yourself first, then look it up.

Hands-on 1: Step by step

30 minutes — no branch switching, fill in every table column while the run is still there

1 **Secure the starting point**

`git status --short` must be empty, otherwise ask us. The reset deletes everything uncommitted.

2 **Run 1: just hand over the ticket**

“Implement what issues/01-products-filterable.md requires. Just get it done.” Don’t intervene.

3 **Have the table filled in for run 1**

Enter the measurement prompt (slide “The Measurement Prompt”). Claude fills in the table: files/lines, decisions, of which recorded, verify. You copy the values into the “Run 1” column.

4 **Reset everything**

First `/clear`, then in the terminal:
`git reset --hard && git clean -fd`

5 **Run 2: clarify first, then implement**

“Don’t write any code yet. List the open points in the ticket with a recommendation and formulate a task: context, goal, constraints, format.” Then: “Adopt all your recommendations and implement the task.”

6 **Measure, commit, add to Miro**

Enter the same measurement prompt again, column “Run 2”. Then commit and enter both columns into the table on the Miro board.

Stuck? The playbook contains all prompts and commands verbatim. Try it yourself first, then look it up.

H2 Hands-on · Project Instructions

Goal: an AGENTS.md that describes our project
30 minutes

REWE DIGITAL

Hands-on 2: Project Instructions

35 minutes

1

Run /init

Start it and let it run to the end.

2

Review the result

What did the model not understand?

3

Add the missing context

Add what only your team knows.

Result: an AGENTS.md that works as a template for your own project

GBOS: The instruction file is called AGENTS.md and describes the how.

Hands-on 2: What to watch out for

35 minutes

Two prompts are enough

One asks what doesn't belong, one what's missing.

Limit every prompt

Max. five findings, one line each, no prose.

Demand evidence

One file:line per convention — otherwise you get made-up rules.

Never rewrite in a single pass

Propose, decide, apply — step by step.

Finally: rename to AGENTS.md and keep @AGENTS.md in the CLAUDE.md

H3 Hands-on · Build Your Own Skill

Goal: a working Skill as a template
35 minutes

REWE DIGITAL

Hands-on 3: Build Your Own Skill

35 minutes

A recurring task from your own daily work — or one of the two suggestions

Create it as a skill and try it out

Result: a template for your own repo

GBOS: shared skills in English, team-specific skills are allowed

Hands-on 3: What to watch out for

35 minutes

- **Start with the description —** it decides whether the skill triggers at all
- **Suggestion A: characterization tests —** try it on the DepositCalculator
- **Suggestion B: clean up an instruction file —** try it on your AGENTS.md
- **Restart Claude Code or /reload-skills —** a new skills directory is not watched
- **Success test:** phrase the task without naming the skill — if it doesn't trigger, the description is the problem, not the instructions

H4 Hands-on · Using a Subagent

Goal: see a delegation in action and when it pays off
25 minutes

REWE DIGITAL

Hands-on 4: Using a Subagent

25 minutes

Task goes out → **Subagent · own context** → **Result comes back**

```
.claude/agents/money-audit.md
---
name: money-audit
description: …
tools: Read, Grep, Glob
model: inherit
---
```

The tools: field is the access list — anything not listed is forbidden

Where does context isolation help — and where does it get in the way?

GBOS: Limit access, dedicated tokens with minimal permissions

REWE DIGITAL

Hands-on 4: What it’s about

25 minutes

The idea: You hand off work to a subagent that runs in its own session with limited permissions. Only the result comes back.

`.claude/agents/<name>.md`

The **`tools:`** line is the access list. Anything not listed there is forbidden. Here: only Read, Grep, Glob.

Delegate the result, not the path: search and check yes, change and decide no.

Done when

the file is entirely in English, tools: is read-only, the call returns file and line, and everything is committed.

1. **Have the subagent created**
   Our suggestion money-audit or your own agent. File entirely in English.
2. **Check the tools: line**
   Only **`Read, Grep, Glob`** allowed. If there is more, have Claude fix the line.
3. **Restart and invoke**
   Restart Claude Code, then invoke with **`@money-audit`** or @<name>. The @ forces the delegation.
4. **Commit**
   Result: a list with file and line. Then commit **`.claude/agents`**.

Stuck? The playbook contains all prompts and commands verbatim. Try it yourself first, then look it up.

Hands-on 4: What you tell Claude

25 minutes

Option A: our suggestion money-audit

1 · Create the subagent

```
“Create a project subagent money-audit that finds every place where this service handles money. Read-only: it may use read, grep and glob, nothing else. It reports file and line for every place where an amount is stored, calculated or returned. Write the file entirely in English.”
```

Check: tools: only Read, Grep, Glob. Then restart.

2 · Invoke, then commit .claude/agents

```
“@money-audit find every place where this service handles money”
```

Option B: your own agent

Have an idea that’s more useful to you? Go for it. For example:

Create

```
“Create a project subagent endpoint-map. It lists all REST endpoints of this service: HTTP method, path, file and line. Only Read, Grep, Glob. File entirely in English.”
```

Invoke

```
“@endpoint-map list all endpoints”
```

For your idea: swap name and task. Stays the same: read-only tools, fixed output format, file in English.

To copy: The prompts for money-audit and the commit are in the playbook verbatim.

H5 Hands-on · Team Configuration

Goal: a settings.json the team can share
25 minutes

REWE DIGITAL

Hands-on 5: What it’s about

25 minutes

The idea: Claude either follows the AGENTS.md or it doesn’t. Claude Code itself enforces the settings.json, no matter what the model intends.

`.claude/settings.json`
Applies to everyone in the repo, lives in Git. Already exists: locks the playbooks. You extend it.

`.claude/settings.local.json`
Applies only to you, is gitignored. Not committed.

Done when
the hook has reformatted, git push was rejected, the playbook lock is in place and only settings.json is committed.

1. **Have the shared file extended**
   Hook: formatter after every Edit/Write. Rule: forbid git push. The three playbook rules stay under deny.
2. **Have the private file created**
   `settings.local.json` with a rule just for you: ./mvnw without asking.
3. **Try it out**
   A wrongly indented comment gets formatted automatically. Claude Code rejects git push without asking.
4. **Check and commit**
   `git diff` only adds lines. Then commit only the settings.json.

Stuck? The playbook contains all prompts and commands verbatim. Try it yourself first, then look it up.

Hands-on 5: What you tell Claude

25 minutes

Configure

1 · Extend the shared file

```
“Extend the existing .claude/settings.json with two
things: a PostToolUse hook on Edit and Write that
runs the formatter, and a permission rule that
forbids git push. Keep existing entries.”
```

Check: the three playbook rules are still under deny.

2 · Create the private file

```
“Now create .claude/settings.local.json with a
permission rule that only affects me: allow ./mvnw
without asking.”
```

Try it out and commit

3a · Trigger the hook

```
“Add a comment to TrainingApplication.java,
deliberately indented wrong.”
```

git diff shows it cleanly indented. Then: git restore src/main

3b · Try a push

```
“Run git push.”
```

Claude Code refuses without asking. If it asks: say no.

4 · Check and commit

```
git diff .claude/settings.json
git add .claude/settings.json && git commit -m "Extend
shared Claude Code settings"
```

To copy: All prompts and commands are in the playbook verbatim.

H6 Hands-on · The Big Block

Goal: a complete run with a reviewed result
45 minutes

REWE DIGITAL

Hands-on 6: What it’s about

45 minutes — ticket **`issues/02-deposit-return.md`**

The idea: Implement a larger ticket according to plan in small, reviewable substeps. You don’t need to understand the code: you check whether each substep does what the plan says and whether the build stays green.

1 Write the plan	2 Review the plan	3 Substeps	4 Accept
Choose model and effort, plan as a file, no code yet	With an empty context, max. 5 findings, then commit	One after the other, flow see below	Against the acceptance criteria in the ticket

3 · Per substep, until the plan is done (after /clear Claude only knows the plan)

`/model /effort` → **Have it implemented** → **Test it yourself** → **Commit code + plan** → **Copy the prompt** → `/clear`

Stuck? The playbook contains all prompts and commands verbatim. Try it yourself first, then look it up.

Hands-on 6: Step by step

45 minutes — ticket **`issues/02-deposit-return.md`**

1 **Model and effort for planning**
`/model` and **`/effort`** choose deliberately and justify the choice in one sentence.

2 **Have the plan written as a file**
`docs/plan-deposit-return.md`, no code yet: execution rules, decisions on the 3 open questions, max. 5 substeps.

3 **Have the plan reviewed**
After **`/clear`** have it reviewed fresh, work in max. 5 findings, commit the plan, **`/clear`**.

4 **Substep by substep**
Model and effort as per plan, implement, **`verify`** green and plan ticked off? Commit code and plan, copy the prompt, **`/clear`**.

5 **Accept**
Claude checks against the acceptance criteria in the ticket, one piece of evidence per criterion (file:line or test name).

Done when
all substeps ticked off and committed, **`verify`** is green and **`POST /api/returns`** returns a 150-cent deposit

Stuck? The playbook contains all prompts and commands verbatim. Try it yourself first, then look it up.